



Launching KelanaAI to the World

Build KelanaAI with Python, Next.js & Amazon Bedrock — Session 11

🕒 Duration: 2 Hours

☁️ Week 4 · Cloud Deployment

🕒 Feature: Publicly Accessible AI Application

Yesterday KelanaAI became a complete conversational AI assistant.

Today we are making it available to the world.



SFIA Competency Card

Session 11 — Launching KelanaAI to the World

FEATURE DELIVERED



Publicly Accessible AI Application

PRIMARY SFIA SKILLS

RELM L2 Release & Deployment

CFMG L2 Configuration Management

ITOP L2 IT Infrastructure

PROG L2 Programming / Software Development

EVIDENCE — BY END OF SESSION, STUDENTS PRODUCE:

- ✓ Backend deployed to Render
- ✓ Frontend deployed to Vercel
- ✓ PostgreSQL hosted on Neon
- ✓ Environment variables configured
- ✓ Public URL accessible
- ✓ Git commit

✓ production-ready

Learning Objectives

By the end of this session, students will be able to:

01

Explain why deployment is part of software engineering.

Code that only runs on your laptop is unfinished.

02

Deploy a FastAPI application to Render.

Containerized Python backend as a public web service.

03

Deploy a Next.js application to Vercel.

Frontend hosting purpose-built for Next.js.

04

Connect a managed PostgreSQL database.

Neon: serverless Postgres with a connection string.

05

Configure production environment variables.

Never hardcode secrets — use platform env vars.

06

Verify end-to-end communication across all services.

Browser → Vercel → Render → Neon → Bedrock → back.



Product Roadmap

Where we are — Week 4, Cloud Deployment

WEEK 1	WEEK 2	WEEK 3	WEEK 4 TODAY
✓ Trip Summary	✓ PostgreSQL	✓ Trip Dashboard	✓ Conversation Memory
✓ Recommendation Engine	✓ Amazon Bedrock	✓ Authentication	▶ Cloud Deployment
✓ REST API	✓ Next.js Frontend	✓ RAG	✓ Demo Day



Yesterday: KelanaAI became a complete conversational AI assistant. **Today:** we make it available to the world.



Story — From localhost to the World

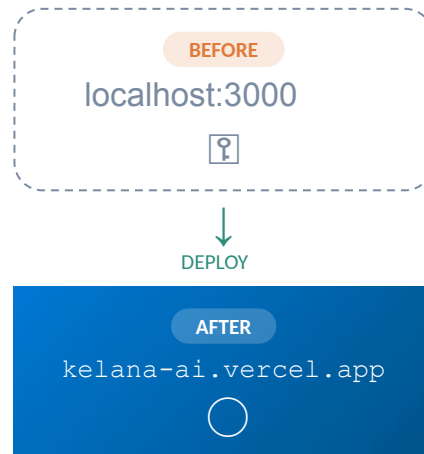
Until now, only developers could use KelanaAI.



Real users do not have your laptop.

They need a website, a public URL, and reliable access.

Today we'll publish KelanaAI so anyone with a browser can use it.



Technical Lesson

Eight parts — from production architecture to deployment checklist.

01

Prod Architecture

02

Env Variables

03

Deploy Database

04

Deploy Backend

05

Deploy Frontend

06

E2E
Testing

07

Common Issues

08

Checklist

Each service has one responsibility.

Part 1 — Production Architecture

Each service has one responsibility. This architecture is commonly used by startups.

ARCHITECTURE

Five services,
five responsibilities.

KEY PRINCIPLE

Each service has one responsibility. This architecture is commonly used by startups.



User
browser

any device, anywhere



Vercel (Next.js)
frontend hosting

kelana-ai.vercel.ap
p



Render (FastAPI)
backend API

kelana-ai-api.onrender.com



Neon PostgreSQL
managed database

ep-xxx.neon.tech



Amazon Bedrock
LLM inference

aws.amazon.com/bedrock





Part 2 — Environment Variables

Never hardcode secrets. Use .env — dev and production have different configurations.

.env

Database

DATABASE_URL=postgresql://user:pass@host/db

AWS Bedrock

AWS_REGION=us-east-1

AWS_ACCESS_KEY_ID=AKIAxxxxxxxxxxxx

AWS_SECRET_ACCESS_KEY=xxxxxxxxxxxxxxxxxxxxxxxx

BEDROCK_MODEL_ID=anthropic.claude-3-sonnet

Frontend

NEXT_PUBLIC_API_URL=https://kelana-ai-api.onrender.com

NEVER HARDCODE



Never commit these to Git:

- ✗ API Keys
- ✗ Database Passwords
- ✗ AWS Credentials

PRINCIPLE



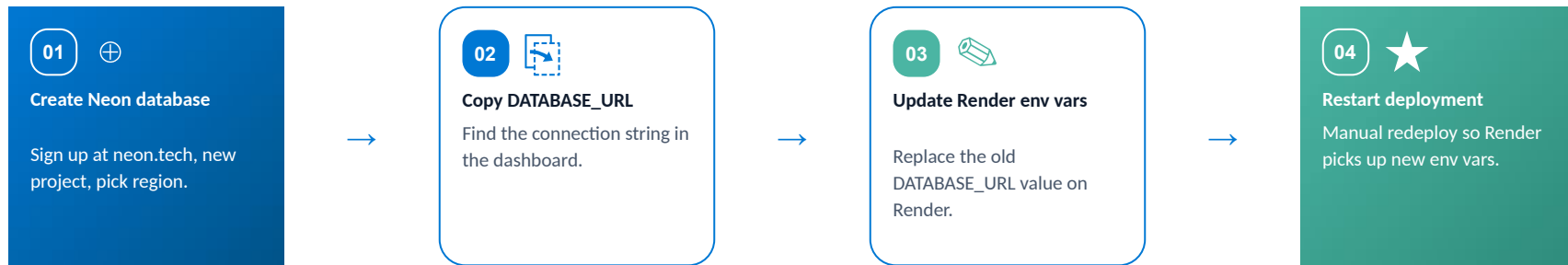
Dev ≠ Production

Your local .env uses localhost and a local Postgres. Production uses public URLs and a managed database. Configure each environment separately on the hosting platform.



Part 3 — Deploy Database on Neon

Managed PostgreSQL — copy the connection string into Render.



VERIFY

Trip data is stored successfully

Generate a trip from the frontend, then check the Neon SQL editor — you should see the row.

id	destination	created_at
1	Japan	2025-07-03 09:14
2	Singapore	2025-07-03 09:22



Part 4 — Deploy Backend to Render

Push to GitHub, create a Render service, configure, deploy, verify.

01

Push latest code to GitHub

Make sure main branch is up to date with all Session 10 commits.



02

Create a Render service

New → Web Service → connect your GitHub repo.



03

Configure Python version + build/start commands

Build: pip install -r requirements.txt · Start: uvicorn main:app --host 0.0.0.0 --port \$PORT



04

Add environment variables

DATABASE_URL, AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, BEDROCK_MODEL_ID.



05

Deploy and wait for build

First build takes ~3 minutes. Watch the logs.



VERIFY



Confirm the API is live

`https://kelana-ai-api.onrender.com`

CHECKLIST

- ✓ Open Swagger UI at /docs
- ✓ Hit the /health endpoint
- ✓ Confirm no 500 errors in logs

Render free tier sleeps after 15 min idle — first request after sleep takes ~30s.



ALKADEMI - MAIN

SESSION 11 · PART 4

Part 4 — Deploy Backend to FastApiCloud

Push to GitHub, create a Render service, configure, deploy, verify.

01

Push latest code to GitHub

Make sure main branch is up to date with all Session 10 commits.



02

Create a FastApiCloud service

New Web App → connect your GitHub repo.



03

Setup root directory backend

Build: pip install -r requirements.txt · Start: uvicorn main:app --host 0.0.0.0 --port \$PORT



04

Add environment variables

DATABASE_URL, AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, BEDROCK_MODEL_ID.



05

Deploy and wait for build

First build takes ~3 minutes. Watch the logs.



VERIFY



Confirm the API is live

`https://kelana-ai-api.onrender.com`

CHECKLIST

- ✓ Open Swagger UI at /docs
- ✓ Hit the /health endpoint
- ✓ Confirm no 500 errors in logs

Render free tier sleeps after 15 min idle — first request after sleep takes ~30s.



ALKADEMI - MAIN

SESSION 11 · PART 5

Part 5 — Deploy Frontend to Vercel

Import the repo, set the API URL, deploy. Every Git push can trigger a redeploy.

01

Import repository into Vercel

Sign in with GitHub, click New Project, select your frontend repo.



02

Configure NEXT_PUBLIC_API_URL

Set it to your Render backend URL: <https://kelana-ai-api.onrender.com>



03

Deploy

Vercel auto-detects Next.js. Build takes ~60 seconds.



04

Enable auto-deploy on Git push

Every push to main triggers a new deployment automatically.



VERIFY



Open the public URL

<https://kelana-ai.vercel.app>

CHECKLIST

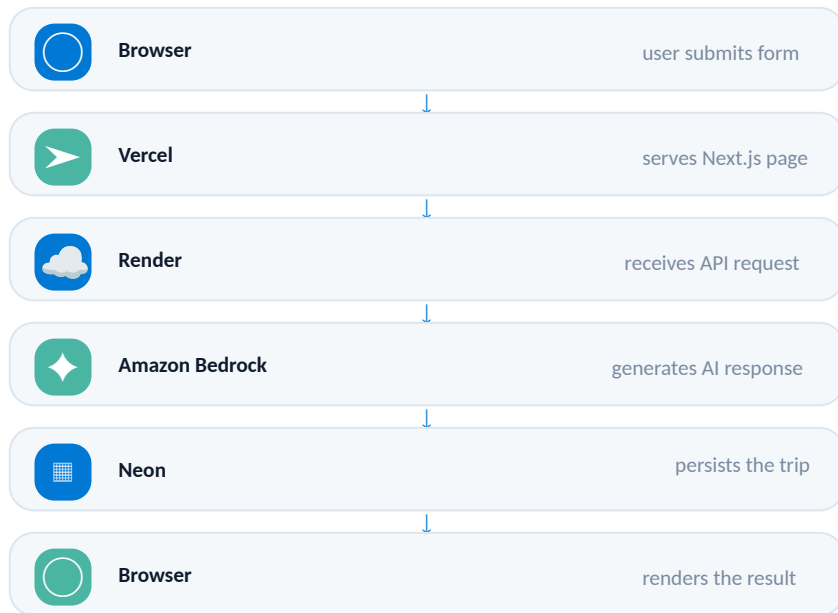
- ✓ Page loads without 404
- ✓ Login form submits to Render
- ✓ No CORS errors in console

Vercel assigns a free *.vercel.app domain. Custom domains can be added later.



Part 6 — End-to-End Testing

Generate a trip end-to-end. Every service in the chain must respond.



VERIFY — EVERY FEATURE WORKS



- ✓ Login
JWT auth round-trips through Render
- ✓ Dashboard
pulls trips from Neon
- ✓ AI Recommendation
Bedrock returns a fresh itinerary
- ✓ Conversation History
messages persist across reloads
- ✓ No console errors in the browser



Everything should work.

Part 7 — Common Deployment Issues

Troubleshooting is part of software engineering. Know the usual suspects.

CORS



CORS Errors

Browser blocks cross-origin requests from Vercel to Render.

Fix: Add Vercel domain to Render CORS allow-list.

CONFIG



Missing Env Vars

Render or Vercel throws 500 because an env var is unset.

Fix: Re-check platform env var dashboard.

URL



Incorrect API URLs

Frontend calls localhost:8000 in production.

Fix: Set `NEXT_PUBLIC_API_URL` to the Render URL.

DB



DB Connection Failures

Render cannot reach Neon — wrong connection string or IP not allow-listed.

Fix: Verify `DATABASE_URL` format and Neon IP allow-list.

AWS



Expired AWS Credentials

Bedrock returns 403 — IAM keys rotated but not updated.

Fix: Generate new IAM keys, update Render env vars.



Troubleshooting is part of software engineering.

Read the logs, isolate the layer, fix one thing at a time.



Part 8 — Deployment Checklist

Professional teams verify releases using checklists. Run through every item before announcing go-live.



Backend works

Render service is up, Swagger loads, /health returns 200.



Frontend works

Vercel page loads, no JS errors in console.



Database connected

Neon shows active connection, queries return rows.



Authentication works

Login + JWT round-trip succeed end-to-end.



AI responses generated

Bedrock returns a valid itinerary for a sample trip.



Environment variables secured

No secrets in Git, all env vars set on hosting platforms.



HTTPS enabled

Vercel and Render both serve over HTTPS by default.



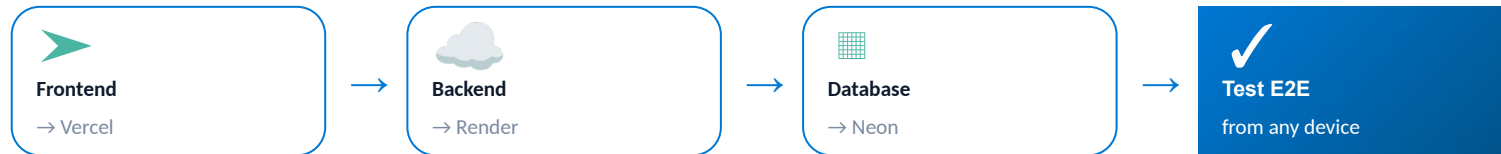
Professional teams verify releases using checklists.

Print this. Tape it to your monitor. Use it on every deploy.



Hands-on Lab

Deploy all three services. Test the complete application from anywhere.



Expected result

· <https://your-kelana-ai.vercel.app>



Register



Login



Generate itinerary



Chat with AI



Retrieve trip history

Test from your phone too — KelanaAI should work from anywhere.

Challenge

Hand your app to a real human. Watch them use it. Record what breaks.

CORE CHALLENGE — CLASSMATE BETA TEST



Share your deployed app with a classmate

Send them your Vercel URL. Ask them to use KelanaAI end-to-end and report any bugs.

ASK THEM TO:

- ✓ Register an account
- ✓ Login
- ✓ Generate an itinerary
- ✓ Ask the AI questions
- ✓ Continue an existing conversation

Record any bugs they discover — fix them tonight.

BONUS



Add a custom favicon and logo

Make KelanaAI look like a real product, not a default Next.js starter.

EXAMPLES

- 📁 favicon.ico in /public
- 📁 Logo in header
- ↗️ OG image for social shares

Hint: use Figma or any free logo generator.



Git Checkpoint

Commit today's deployment config. KelanaAI is now accessible beyond your own computer.

```
bash - kelana-ai

# stage today's deployment work
→ git add .

# commit with a descriptive message
→ git commit -m "Deploy KelanaAI to production"

# push to trigger Vercel auto-deploy
→ git push
```



Today KelanaAI became accessible beyond your own computer.

Tag your commit so it's easy to find later: [git tag session-11](#)

Mini Quiz

Quick check — discuss with your neighbour, then we'll review.

Q1



Why shouldn't AWS credentials be committed to Git?

Hint: Think about who can see public repo history.

Q2



What is the purpose of environment variables?

Hint: Different configs for different environments.

Q3



Why do frontend and backend use separate hosting platforms?

Hint: Each platform is optimized for a different workload.

Q4



What happens after pushing new code to GitHub?

Hint: Vercel is watching your repo.



Discuss for 5 minutes. We'll review answers together.

Homework & What's Next

Polish the production app tonight — and prep for Demo Day.

Homework — Improve Your Production App



Add these five production polish items:

01



Custom application icon

02



Better error page (404 + 500)

03



Loading screen

04



About page

05



README deployment instructions



Commit and push.

NEXT SESSION — 12 · FINAL

Demo Day

Today KelanaAI became a real cloud application. Friends, family, recruiters, and potential employers can now access it from anywhere.

Tomorrow we complete the bootcamp:

- Present KelanaAI like a professional engineering team
- Review the architecture you've built
- Learn what it means to be an AI Native Software Engineer

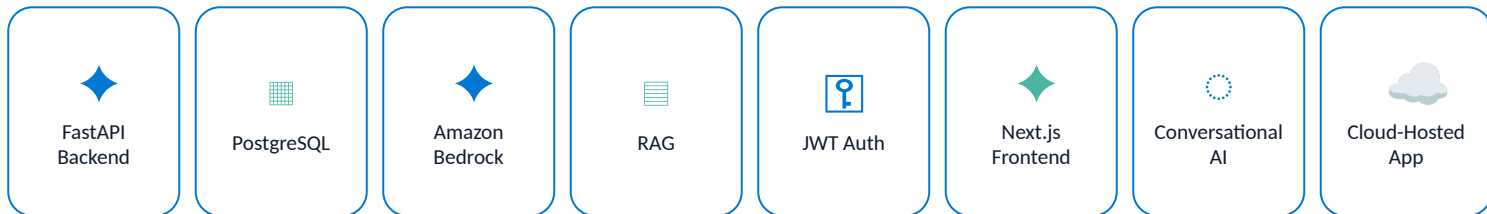
Bring your demo script and your best energy.



MILESTONE ACHIEVED

KelanaAI is now a publicly accessible AI application

You crossed one of the biggest milestones in software engineering. KelanaAI is no longer a local project — it is a real cloud application.



This is remarkably close to the architecture used by many modern AI startups.

Your next challenge isn't writing more code — it's demonstrating, explaining, and confidently presenting what you've built.